

Helix VPN — Aurora OS Client Specification

Helix VPN — Aurora OS Client Specification

Comprehensive Technical Specification for Aurora OS (Sailfish OS) VPN Client

Version: 1.0

Date: July 2025

Status: Technical Specification — Draft

Platform: Aurora OS 4.x / 5.x (Sailfish OS derivative)

Architecture: Qt6/QML + C++ + Rust Core via FFI

Distribution: OpenRepos / Aurora Store (Harbor)

Table of Contents

1. [Aurora OS Overview](#)
 2. [Architecture](#)
 3. [VPN Integration](#)
 4. [Rust Core Integration](#)
 5. [UI Implementation](#)
 6. [Platform-Specific Features](#)
 7. [Build System](#)
 8. [Comparison with Other Platforms](#)
 9. [Implementation Phases](#)
-

1. Aurora OS Overview

1.1 What is Aurora OS

Aurora OS (auroraos.ru) is a Russian mobile operating system based on Sailfish OS, developed by Open Mobile Platform (OMP). It is designed primarily for the Russian market as part of the country's digital sovereignty initiative, providing an alternative to Android and iOS for government and enterprise use.

Key characteristics:

- **Origin:** Forked from Sailfish OS by Jolla (Finland)
- **Maintainer:** Open Mobile Platform (OMP), Russia
- **Current Version:** 4.x / 5.x series (aligned with Sailfish OS releases)
- **License:** Proprietary OS with open-source components
- **Target Market:** Russian government, enterprise, and consumer sectors

1.2 Market Context and Target Users

Segment	Description	Device Examples
Government	Federal and regional government employees	F+ R570E, Aquarius CMP NS
Enterprise	Corporate fleets, secure communications	Custom OEM devices
Consumer	Privacy-conscious Russian consumers	INOI, Aquarius, F+ devices
Specialized	Industrial, medical, logistics	Ruggedized handhelds

Aurora OS devices are primarily mid-range ARM-based smartphones with constrained hardware compared to flagship Android/iOS devices. This makes performance optimization critical.

1.3 Technical Foundation

Aurora OS inherits the Sailfish OS technology stack:

UI Layer	
Qt6 / QML + Sailfish Silica	
Middleware	
ConnMan (network)	oFono (telephony)
PulseAudio (audio)	systemd (init)

Core OS
Mer Core (Linux-based) / Nemo Mobile
Wayland compositor (Lipstick)
Kernel
Linux Kernel (adapted per-device)

Key components:

- **ConnMan**: Connection Manager — handles all network connectivity including VPN
- **Wayland**: Display server protocol (Lipstick compositor)
- **Qt6/QML**: Native UI framework
- **Silica**: Sailfish-specific QML UI component library
- **RPM**: Package management (no APK, no DEB)
- **Sailjail**: Sandboxing based on Firejail
- **oFono**: Telephony stack

1.4 Platform Capabilities and Limitations

Capability	Status	Notes
Native VPN (WireGuard)	Yes	Via ConnMan VPN plugin since Sailfish OS 5.0
Custom VPN protocols	Limited	Requires ConnMan plugin development
Background execution	Restricted	Sailjail sandboxing applies
System notifications	Yes	Via nemo-qml-plugin-notifications
System tray / Status icon	No	Uses Cover (active frame) instead
Auto-start on boot	Limited	Via systemd user services
D-Bus access	Restricted	Must declare in .desktop file permissions
File system access	Sandboxed	Requires explicit permissions
Qt Quick Controls	Forbidden	Not allowed in Harbor validation
Silica UI components	Required	Must use for Harbor acceptance

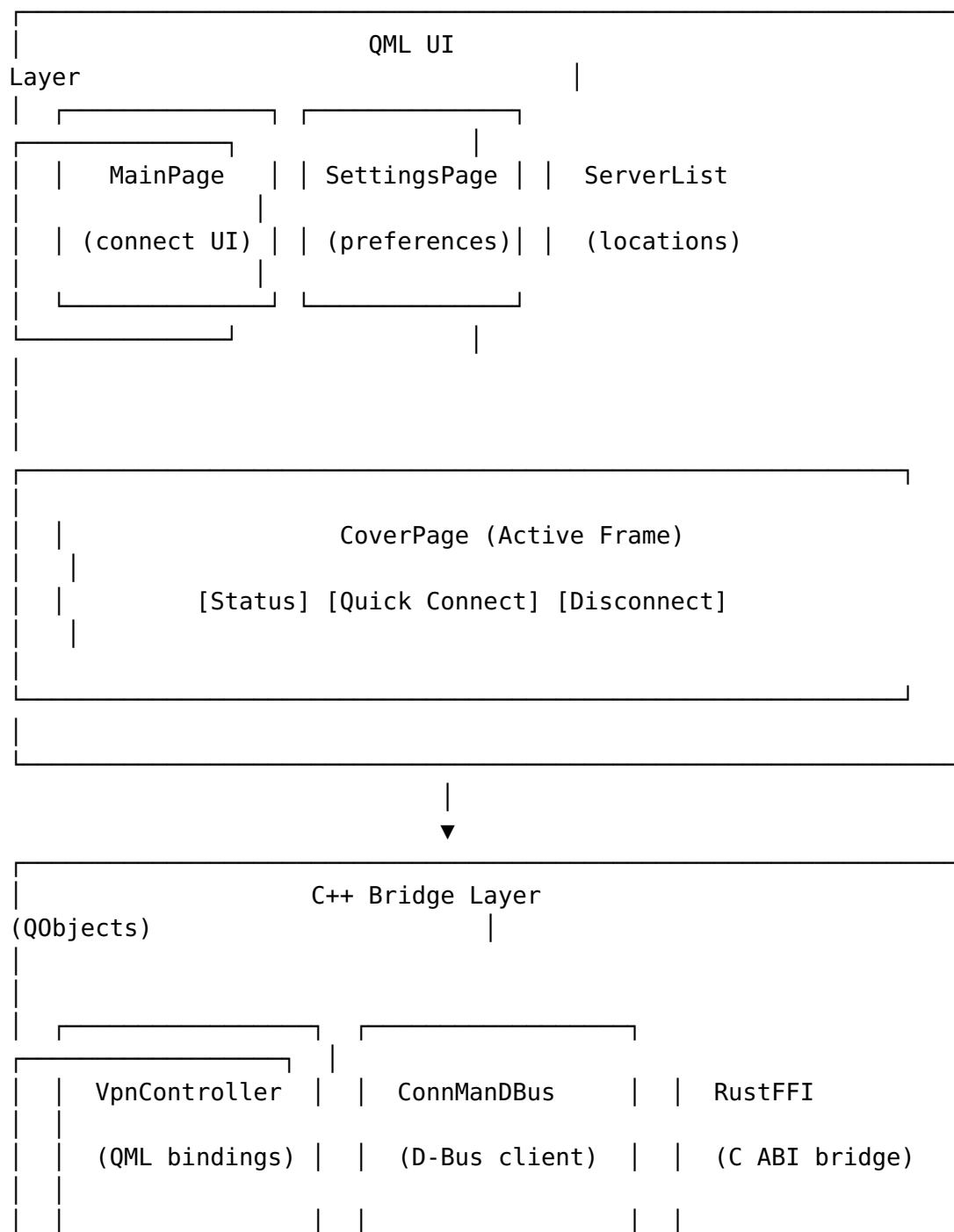
Critical Limitations:

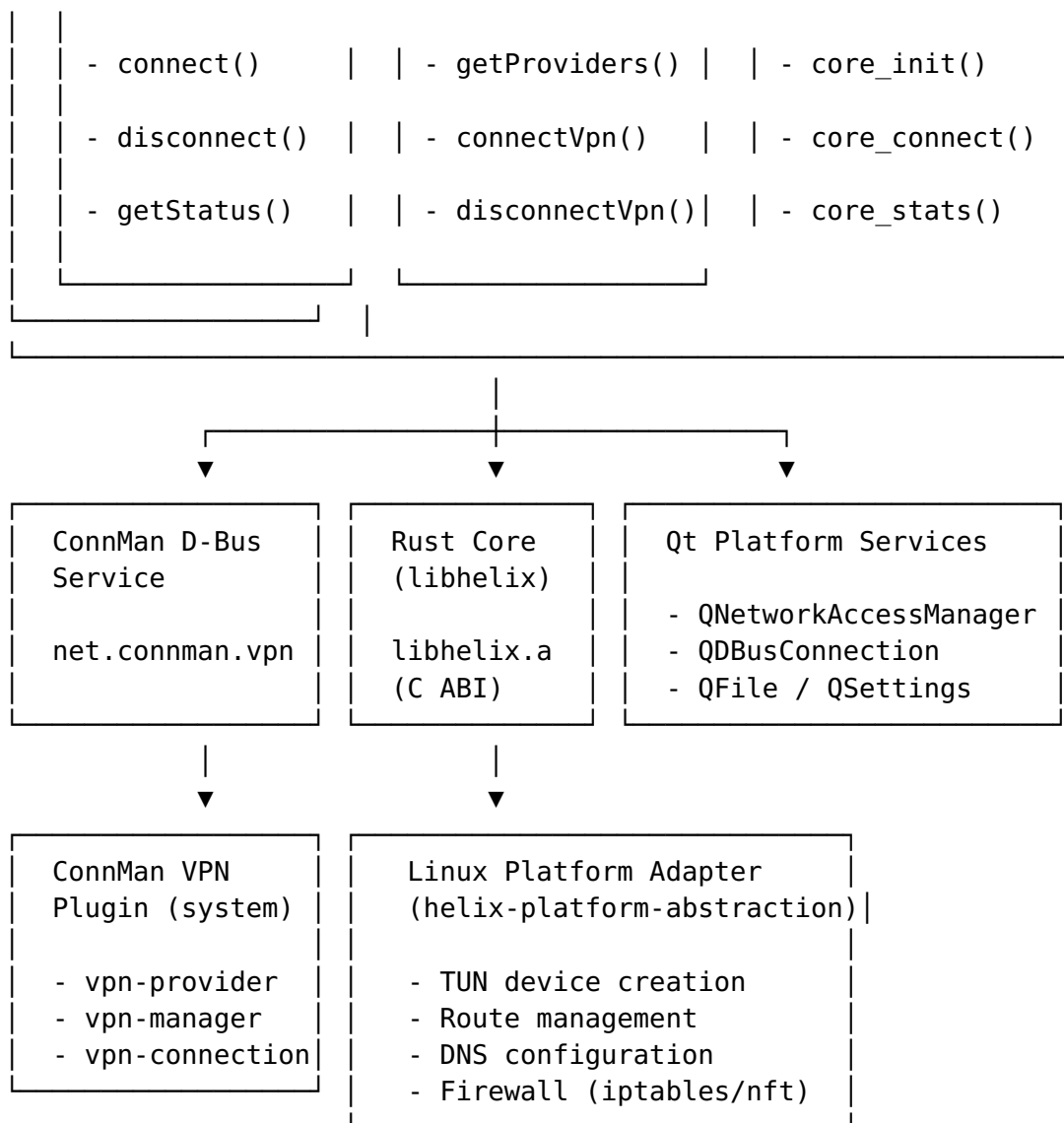
1. **No system tray** — Aurora OS uses "Covers" (active frames) for background app status
2. **Sailjail sandboxing** — Apps must declare D-Bus and network permissions

3. **Qt Quick Controls forbidden** — Must use Sailfish Silica components exclusively
 4. **Hardware constraints** — Target devices typically have 2-4 GB RAM, mid-range ARM CPUs
 5. **No Google Play Services** — Distribution via OpenRepos or Aurora Store only
-

2. Architecture

2.1 High-Level Architecture





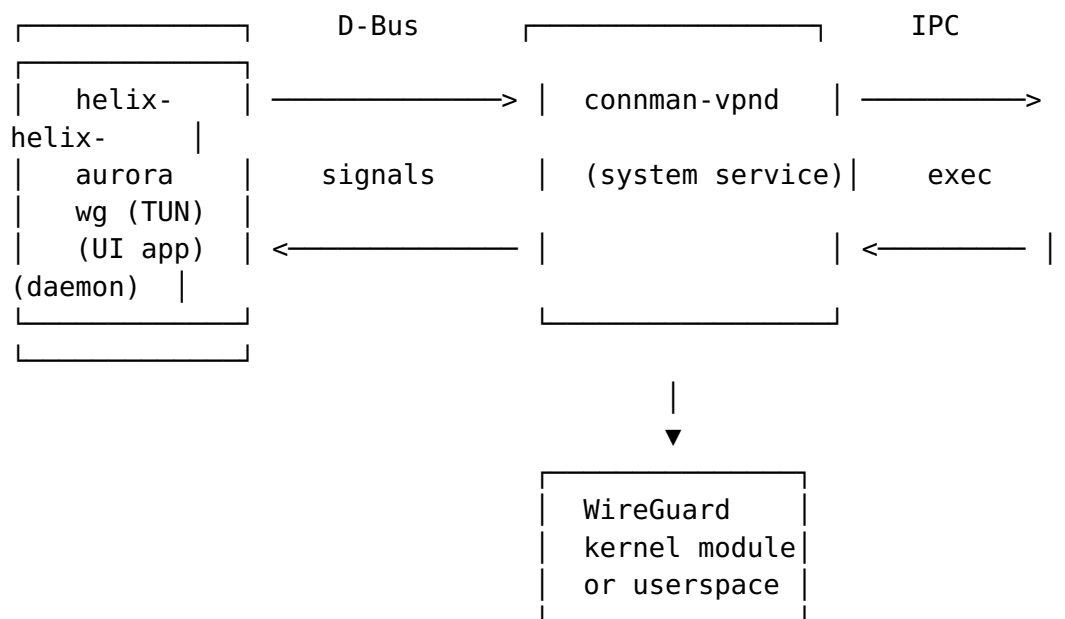
2.2 Qt6/QML + C++ + Rust Core via FFI

The Aurora OS client follows the same architectural pattern as other Helix platforms:

1. **Rust Core** (libhelix_aurora.so): Compiled as a shared library, contains all VPN protocol logic, encryption, and platform abstraction
2. **C++ Bridge** (vpncontroller.h/cpp): QObject-derived classes exposed to QML via Qt's meta-object system
3. **QML UI**: Sailfish Silica components consuming C++ bridge objects

2.3 Integration with ConnMan

ConnMan is the connection manager on Aurora OS. All VPN connectivity goes through ConnMan's VPN subsystem:



2.4 Sailfish Silica UI Components

All UI must use Sailfish Silica components. Key types:

Silica Component	Purpose	Standard Qt Equivalent
ApplicationWindow	Top-level app container	QQuickWindow
Page	Screen content container	Item
PageStack	Navigation stack	StackView
CoverBackground	Background app cover	N/A
PullDownMenu	Top pulley menu	Menu
SilicaListView	Styled list view	ListView
ListItem	List row with press effect	Item
Label	Styled text label	Text
Button	Styled button	Button (forbidden)
IconButton	Icon-only button	ToolButton
TextSwitch	Toggle switch	Switch
Slider	Value slider	Slider
ValueButton	Value selector button	N/A
Dialog	Modal dialog	Dialog
RemorseItem	Undo action	Snackbar
Theme		N/A

Silica Component	Purpose	Standard Qt Equivalent
	Color/font/size constants	

2.5 RPM Packaging for OpenRepos

Distribution is via RPM packages:

- **OpenRepos:** Community-driven app store (openrepos.net)
- **Harbor:** Official Aurora Store (requires validation)
- **Manual:** Direct RPM installation

3. VPN Integration

3.1 ConnMan VPN Plugin Architecture

ConnMan manages VPN connections through a plugin architecture. The VPN subsystem consists of:

- **connman-vpnd:** VPN daemon (runs as system service)
- **VPN Provider:** Represents a VPN configuration (WireGuard, OpenVPN, etc.)
- **VPN Manager:** Manages the list of VPN providers
- **VPN Connection:** Active VPN connection state

The D-Bus service name for VPN: `net.connman.vpn`

3.2 D-Bus Interface for VPN Control

ConnMan VPN exposes these D-Bus interfaces:

```
Service:    net.connman.vpn
Object:     /
Interface:  net.connman.vpn.Manager
```

Methods:

```
GetConnections() -> array(object_path, dict) - List all VPN
connections
CreateConnection(dict, object_path) -> object_path - Create new
connection
RemoveConnection(object_path) - Remove a connection
RegisterAgent(object_path) - Register a VPN agent
UnregisterAgent(object_path) - Unregister agent
```

Signals:

```
ConnectionAdded(object_path, dict) - New connection created
```

ConnectionRemoved(object_path)	– Connection removed
PropertyChanged(string, variant)	– Manager property changed

Service: net.connman.vpn
Object: /connection/<id>
Interface: net.connman.vpn.Connection

Methods:

- GetProperties() -> dict – Get connection properties
- SetProperty(string, variant) – Set a property
- ClearProperty(string) – Clear a property
- Connect() – Connect this VPN
- Connect2(string) – Connect with D-Bus sender proxy
- Disconnect() – Disconnect this VPN

Signals:

- PropertyChanged(string, variant) – Property changed

Properties:

- State – "idle", "failure", "association", "configuration", "ready", "disconnect"
- Type – VPN type ("wireguard", "openvpn", etc.)
- Name – Display name
- Host – VPN server host
- Domain – Domain name
- Immutable – Whether config is read-only
- SplitRouting – Split tunneling enabled
- Index – Tunnel interface index
- IPv4 – IPv4 configuration dict
- IPv6 – IPv6 configuration dict
- Nameservers – DNS server array
- UserRoutes – User-defined routes
- ServerRoutes – Server-pushed routes

3.3 Qt D-Bus Integration Code

```
// connmandbus.h – ConnMan D-Bus client for Aurora OS
#ifndef CONNMANDDBUS_H
#define CONNMANDDBUS_H

#include <QObject>
#include <QDBusInterface>
#include <QDBusConnection>
#include <QDBusMessage>
#include <QDBusArgument>
#include <QVariantMap>
#include <QStringList>

class ConnManVpnManager : public QObject
{
    Q_OBJECT
```



```

    Q_PROPERTY(bool available READ available NOTIFY
                availabilityChanged)
    Q_PROPERTY(QVariantMap connections READ connections NOTIFY
                connectionsChanged)

public:
    explicit ConnManVpnManager(QObject *parent = nullptr);

    bool available() const;
    QVariantMap connections() const;

    Q_INVOKABLE void refreshConnections();
    Q_INVOKABLE void connectVpn(const QString &connectionPath);
    Q_INVOKABLE void disconnectVpn(const QString &connectionPath);
    Q_INVOKABLE QVariantMap getConnectionProperties(const QString
        &connectionPath);

signals:
    void availabilityChanged();
    void connectionsChanged();
    void connectionStateChanged(const QString &path, const
        QString &state);
    void error(const QString &message);

private slots:
    void onConnectionAdded(const QDBusObjectPath &path, const
        QVariantMap &properties);
    void onConnectionRemoved(const QDBusObjectPath &path);
    void onPropertyChanged(const QString &name, const
        QDBusVariant &value);

private:
    QDBusInterface *m_manager = nullptr;
    QVariantMap m_connections;
    bool setupDBus();
};

#endif // CONNMANDDBUS_H

// connmandbus.cpp
#include "connmandbus.h"
#include <QDBusConnectionInterface>
#include <QDebug>

static const QString CONNMAN_VPN_SERVICE = "net.connman.vpn";
static const QString CONNMAN_VPN_PATH = "/";
static const QString CONNMAN_VPN_MANAGER =
    "net.connman.vpn.Manager";
static const QString CONNMAN_VPN_CONNECTION =
    "net.connman.vpn.Connection";

ConnManVpnManager::ConnManVpnManager(QObject *parent) :
    QObject(parent)

```

```

{
    if (setupDBus()) {
        refreshConnections();
    }
}

bool ConnManVpnManager::setupDBus()
{
    QDBusConnection systemBus = QDBusConnection::systemBus();
    if (!systemBus.isConnected()) {
        qWarning() << "Cannot connect to system D-Bus";
        return false;
    }

    // Check if connman-vpn is available
    QDBusConnectionInterface *iface = systemBus.interface();
    if (!iface->isServiceRegistered(CONNMAN_VPN_SERVICE)) {
        qWarning() << "ConnMan VPN service not registered";
        return false;
    }

    m_manager = new QDBusInterface(
        CONNMAN_VPN_SERVICE,
        CONNMAN_VPN_PATH,
        CONNMAN_VPN_MANAGER,
        systemBus,
        this
    );

    if (!m_manager->isValid()) {
        qWarning() << "Failed to create ConnMan VPN manager
        interface:"
            << m_manager->lastError().message();
        return false;
    }

    // Monitor connection signals
    systemBus.connect(CONNMAN_VPN_SERVICE, QString(),
        CONNMAN_VPN_MANAGER, "ConnectionAdded",
        this,
        SLOT(onConnectionAdded(QDBusObjectPath, QVariantMap)));
    systemBus.connect(CONNMAN_VPN_SERVICE, QString(),
        CONNMAN_VPN_MANAGER, "ConnectionRemoved",
        this,
        SLOT(onConnectionRemoved(QDBusObjectPath)));

    emit availabilityChanged();
    return true;
}

void ConnManVpnManager::connectVpn(const QString &connectionPath)
{

```

```

QDBusInterface connection(
    CONNMAN_VPN_SERVICE,
    connectionPath,
    CONNMAN_VPN_CONNECTION,
    QDBusConnection::systemBus()
);

QDBusMessage reply = connection.call("Connect");
if (reply.type() == QDBusMessage::ErrorMessage) {
    emit error(QString("Failed to connect:
        %1").arg(reply.errorMessage()));
}
}

void ConnManVpnManager::disconnectVpn(const QString
    &connectionPath)
{
    QDBusInterface connection(
        CONNMAN_VPN_SERVICE,
        connectionPath,
        CONNMAN_VPN_CONNECTION,
        QDBusConnection::systemBus()
    );

    QDBusMessage reply = connection.call("Disconnect");
    if (reply.type() == QDBusMessage::ErrorMessage) {
        emit error(QString("Failed to disconnect:
            %1").arg(reply.errorMessage()));
    }
}

QVariantMap ConnManVpnManager::getConnectionProperties(const
    QString &connectionPath)
{
    QDBusInterface connection(
        CONNMAN_VPN_SERVICE,
        connectionPath,
        CONNMAN_VPN_CONNECTION,
        QDBusConnection::systemBus()
    );

    QDBusMessage reply = connection.call("GetProperties");
    if (reply.type() == QDBusMessage::ReplyMessage &&
        reply.arguments().count() > 0) {
        return qdbus_cast<QVariantMap>(reply.arguments().first());
    }
    return QVariantMap();
}

```

3.4 VPN Plugin Registration

For Helix VPN to work with ConnMan, we register a WireGuard VPN configuration:

```
// vpnconfigurator.cpp – Create/manage WireGuard VPN
// configurations in ConnMan

bool VpnConfigurator::createWireGuardConnection(
    const QString &name,
    const QString &serverHost,
    const QString &privateKey,
    const QString &publicKey,
    const QString &presharedKey,
    const QString &address,
    const QString &dns,
    int listenPort,
    int mtu)
{
    QVariantMap config;
    config["Type"] = "wireguard";
    config["Name"] = name;
    config["Host"] = serverHost;
    config["Domain"] = "helix.vpn";
    config["WireGuard.Address"] = address;
    config["WireGuard.ListenPort"] = QString::number(listenPort);
    config["WireGuard.PrivateKey"] = privateKey;
    config["WireGuard.PublicKey"] = publicKey;
    config["WireGuard.PresharedKey"] = presharedKey;
    config["WireGuard.MTU"] = QString::number(mtu);
    config["WireGuard.DNS"] = dns;

    // Mark as immutable (managed by Helix app)
    config["Immutable"] = true;

    QDBusMessage reply = m_manager->call("CreateConnection",
        config, QDBusObjectPath("/"));
    if (reply.type() == QDBusMessage::ErrorMessage) {
        qWarning() << "Failed to create connection:" <<
            reply.errorMessage();
        return false;
    }
    return true;
}
```

3.5 Integration with System Settings

Aurora OS provides system VPN settings at **Settings > System > VPN**. Helix VPN registers as a ConnMan WireGuard provider, making configurations visible and manageable from the system settings as well as from the Helix app.

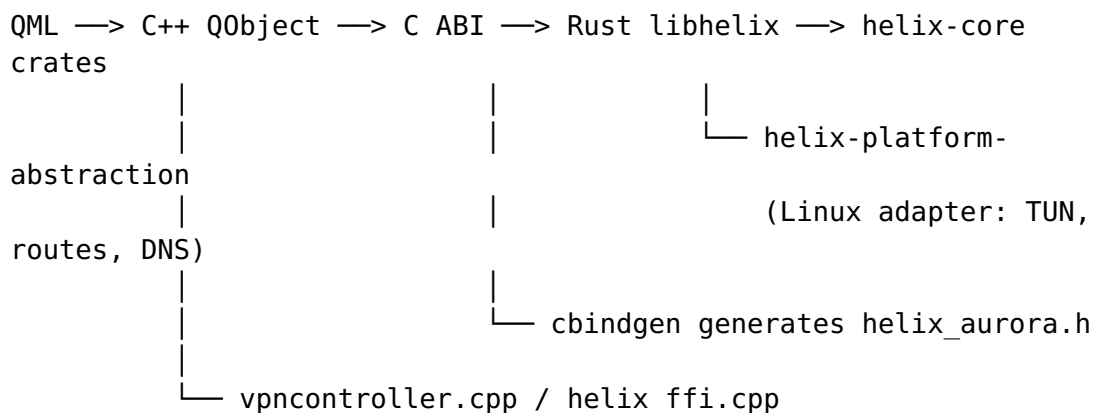
The ConnMan configuration files are stored at:

- /var/lib/connman-vpn/ — VPN configuration files
 - /var/lib/connman/ — Main ConnMan service configs
-

4. Rust Core Integration

4.1 Architecture Overview

The Rust core (helix-core) is compiled as a shared library (libhelix_aurora.so) and loaded by the C++ bridge at runtime. The integration uses direct C FFI with no intermediate binding layer.



4.2 Rust FFI Interface (cbindgen)

```
// crates/helix-core-api/src/ffi_aurora.rs
// Rust side – C ABI exports for Aurora OS

use std::ffi::{c_char, c_int, c_void, CStr, CString};
use std::net::IpAddr;
use std::sync::Arc;
use tokio::runtime::Runtime;

/// Opaque handle to the Helix core
pub struct HelixCore {
    runtime: Runtime,
    engine: Arc<helix_vpn_engine::VpnEngine>,
    config: helix_vpn_engine::Config,
}

/// Connection status callback type
pub type ConnectionCallback = extern "C" fn(state: c_int, info:
    *const c_char, user_data: *mut c_void);

/// Statistics callback type
pub type StatsCallback = extern "C" fn(bytes_in: u64, bytes_out:
    u64, duration_secs: u64, user_data: *mut c_void);
```

```

// Status constants
pub const HELIX_STATE_DISCONNECTED: c_int = 0;
pub const HELIX_STATE_CONNECTING: c_int = 1;
pub const HELIX_STATE_CONNECTED: c_int = 2;
pub const HELIX_STATE_DISCONNECTING: c_int = 3;
pub const HELIX_STATE_ERROR: c_int = 4;
pub const HELIX_STATE_RECONNECTING: c_int = 5;

/// Initialize the Helix core
///
/// # Safety
/// config_json must be a valid null-terminated UTF-8 string
#[no_mangle]
pub unsafe extern "C" fn helix_init(config_json: *const c_char)
    -> *mut HelixCore {
    if config_json.is_null() {
        return std::ptr::null_mut();
    }

    let config_str = match CStr::from_ptr(config_json).to_str() {
        Ok(s) => s,
        Err(_) => return std::ptr::null_mut(),
    };

    let config: helix_vpn_engine::Config = match
        serde_json::from_str(config_str) {
            Ok(c) => c,
            Err(_) => return std::ptr::null_mut(),
        };

    let runtime = match Runtime::new() {
        Ok(rt) => rt,
        Err(_) => return std::ptr::null_mut(),
    };

    let engine = runtime.block_on(async {
        helix_vpn_engine::VpnEngine::new(config.clone()).await
    });

    let engine = match engine {
        Ok(e) => Arc::new(e),
        Err(_) => return std::ptr::null_mut(),
    };

    let core = Box::new(HelixCore {
        runtime,
        engine,
        config,
    });

    Box::into_raw(core)

```

```

}

/// Free the Helix core
#[no_mangle]
pub unsafe extern "C" fn helix_free(core: *mut HelixCore) {
    if !core.is_null() {
        let _ = Box::from_raw(core);
    }
}

/// Connect to a VPN server
///
/// # Safety
/// server_id must be a valid null-terminated UTF-8 string
#[no_mangle]
pub unsafe extern "C" fn helix_connect(
    core: *mut HelixCore,
    server_id: *const c_char,
    callback: ConnectionCallback,
    user_data: *mut c_void,
) -> c_int {
    if core.is_null() || server_id.is_null() {
        return -1;
    }

    let core = &*core;
    let server_id = match CString::from_ptr(server_id).to_str() {
        Ok(s) => s.to_string(),
        Err(_) => return -1,
    };

    let engine = core.engine.clone();

    core.runtime.spawn(async move {
        match engine.connect(&server_id).await {
            Ok(info) => {
                let info_json =
                    serde_json::to_string(&info).unwrap_or_default();
                let c_info =
                    CString::new(info_json).unwrap_or_default();
                callback(HELIX_STATE_CONNECTED, c_info.as_ptr(),
                    user_data);
            }
            Err(e) => {
                let err_msg = format!("{\\\"error\\\":\\\"{\\\"}\\\"}", e);
                let c_err =
                    CString::new(err_msg).unwrap_or_default();
                callback(HELIX_STATE_ERROR, c_err.as_ptr(),
                    user_data);
            }
        }
    });
}

```

```

    0
}

/// Disconnect from VPN
#[no_mangle]
pub extern "C" fn helix_disconnect(core: *mut HelixCore) -> c_int
{
    if core.is_null() {
        return -1;
    }

    let core = unsafe { &*core };
    let engine = core.engine.clone();

    core.runtime.spawn(async move {
        let _ = engine.disconnect().await;
    });

    0
}

/// Get current connection state
#[no_mangle]
pub extern "C" fn helix_get_state(core: *mut HelixCore) -> c_int {
    if core.is_null() {
        return HELIX_STATE_ERROR;
    }

    let core = unsafe { &*core };
    core.runtime.block_on(async {
        match core.engine.state().await {
            helix_vpn_engine::State::Disconnected =>
                HELIX_STATE_DISCONNECTED,
            helix_vpn_engine::State::Connecting =>
                HELIX_STATE_CONNECTING,
            helix_vpn_engine::State::Connected =>
                HELIX_STATE_CONNECTED,
            helix_vpn_engine::State::Disconnecting =>
                HELIX_STATE_DISCONNECTING,
            helix_vpn_engine::State::Reconnecting =>
                HELIX_STATE_RECONNECTING,
            _ => HELIX_STATE_ERROR,
        }
    })
}

/// Get connection statistics
#[no_mangle]
pub extern "C" fn helix_get_stats(
    core: *mut HelixCore,
    bytes_in: *mut u64,
    bytes_out: *mut u64,
    duration_secs: *mut u64,

```



```

) -> c_int {
    if core.is_null() || bytes_in.is_null() ||
        bytes_out.is_null() || duration_secs.is_null() {
        return -1;
    }

    let core = unsafe { &*core };
    let stats = core.runtime.block_on(async {
        core.engine.statistics().await
    });

    unsafe {
        *bytes_in = stats.bytes_received;
        *bytes_out = stats.bytes_sent;
        *duration_secs = stats.duration_secs;
    }

    0
}

/// Subscribe to statistics updates (called periodically from C++)
#[no_mangle]
pub extern "C" fn helix_poll_stats(
    core: *mut HelixCore,
    callback: StatsCallback,
    user_data: *mut c_void,
) -> c_int {
    if core.is_null() {
        return -1;
    }

    let core = unsafe { &*core };
    let stats = core.runtime.block_on(async {
        core.engine.statistics().await
    });

    callback(stats.bytes_received, stats.bytes_sent,
        stats.duration_secs, user_data);

    0
}

/// Free a string returned by helix
#[no_mangle]
pub unsafe extern "C" fn helix_free_string(s: *mut c_char) {
    if !s.is_null() {
        let _ = CString::from_raw(s);
    }
}

```

4.3 cbindgen Configuration

cbindgen.toml – Generate C headers from Rust FFI

```
crate = "helix-core-api"
language = "C"
namespace = "helix"
cpp_namespace = "helix"
include_guard = "HELIX_AURORA_H"
```

[parse]

```
parse_deps = false
```

[export]

```
prefix = "helix_"
```

[fn]

```
rename_types = "PascalCase"
```

[defines]

```
"feature = aurora" = "HELIX_AURORA"
```

Generated header (helix_aurora.h):

```
#ifndef HELIX_AURORA_H
#define HELIX_AURORA_H

#ifdef __cplusplus
namespace helix {
extern "C" {
#endif

#define HELIX_STATE_DISCONNECTED 0
#define HELIX_STATE_CONNECTING 1
#define HELIX_STATE_CONNECTED 2
#define HELIX_STATE_DISCONNECTING 3
#define HELIX_STATE_ERROR 4
#define HELIX_STATE_RECONNECTING 5

typedef struct HelixCore HelixCore;

typedef void (*ConnectionCallback)(int state, const char *info,
    void *user_data);
typedef void (*StatsCallback)(uint64_t bytes_in, uint64_t
    bytes_out,
                                uint64_t duration_secs, void
    *user_data);

HelixCore *helix_init(const char *config_json);
void helix_free(HelixCore *core);
int helix_connect(HelixCore *core, const char *server_id,
    ConnectionCallback callback, void *user_data);
int helix_disconnect(HelixCore *core);
```

```

int helix_get_state(HelixCore *core);
int helix_get_stats(HelixCore *core, uint64_t *bytes_in,
                    uint64_t *bytes_out, uint64_t *duration_secs);
int helix_poll_stats(HelixCore *core, StatsCallback callback,
                    void *user_data);
void helix_free_string(char *s);

#ifdef __cplusplus
} // extern "C"
} // namespace helix
#endif

#endif // HELIX_AURORA_H

```

4.4 C++ Bridge Implementation

```

// helixffi.h - C++ wrapper for Rust FFI
#ifndef HELIXFFI_H
#define HELIXFFI_H

#include "helix_aurora.h"
#include <QObject>
#include <QString>
#include <QTimer>
#include <QDateTime>

class HelixFfi : public QObject
{
    Q_OBJECT
    Q_PROPERTY(int connectionState READ connectionState NOTIFY
                connectionStateChanged)
    Q_PROPERTY(QString serverName READ serverName NOTIFY
                connectionInfoChanged)
    Q_PROPERTY(QString serverLocation READ serverLocation NOTIFY
                connectionInfoChanged)
    Q_PROPERTY(QString ipAddress READ ipAddress NOTIFY
                connectionInfoChanged)
    Q_PROPERTY(quint64 bytesReceived READ bytesReceived NOTIFY
                statsUpdated)
    Q_PROPERTY(quint64 bytesSent READ bytesSent NOTIFY
                statsUpdated)
    Q_PROPERTY(quint64 connectionDuration READ connectionDuration
                NOTIFY statsUpdated)
    Q_PROPERTY(QString errorMessage READ errorMessage NOTIFY
                errorOccurred)

public:
    explicit HelixFfi(QObject *parent = nullptr);
    ~HelixFfi();

    int connectionState() const { return m_connectionState; }
    QString serverName() const { return m_serverName; }
    QString serverLocation() const { return m_serverLocation; }

```

```

QString ipAddress() const { return m_ipAddress; }
quint64 bytesReceived() const { return m_bytesReceived; }
quint64 bytesSent() const { return m_bytesSent; }
quint64 connectionDuration() const { return
    m_connectionDuration; }
QString errorMessage() const { return m_errorMessage; }

Q_INVOKABLE bool initialize(const QString &configJson);
Q_INVOKABLE void connectToServer(const QString &serverId);
Q_INVOKABLE void disconnect();
Q_INVOKABLE void refreshStats();

signals:
    void connectionStateChanged();
    void connectionInfoChanged();
    void statsUpdated();
    void errorOccurred();
    void connected(const QString &serverInfo);
    void disconnected();

private:
    static void stateCallback(int state, const char *info, void
        *userData);
    static void statsCallback(uint64_t bytesIn, uint64_t
        bytesOut,
                                uint64_t duration, void *userData);
    void updateState(int state, const QString &info);
    void updateStats(quint64 bytesIn, quint64 bytesOut, quint64
        duration);
    void startStatsPolling();
    void stopStatsPolling();

    helix::HelixCore *m_core = nullptr;
    int m_connectionState = 0;
    QString m_serverName;
    QString m_serverLocation;
    QString m_ipAddress;
    quint64 m_bytesReceived = 0;
    quint64 m_bytesSent = 0;
    quint64 m_connectionDuration = 0;
    QString m_errorMessage;
    QTimer *m_statsTimer = nullptr;
};

#endif // HELIXFFI_H

// helixffi.cpp
#include "helixffi.h"
#include <QJsonDocument>
#include <QJsonObject>
#include <QDebug>

HelixFfi::HelixFfi(QObject *parent) : QObject(parent)

```

```

{
    m_statsTimer = new QTimer(this);
    m_statsTimer->setInterval(2000); // 2 second polling
    connect(m_statsTimer, &QTimer::timeout, this,
            &HelixFfi::refreshStats);
}

HelixFfi::~HelixFfi()
{
    stopStatsPolling();
    if (m_core) {
        helix::helix_free(m_core);
    }
}

bool HelixFfi::initialize(const QString &configJson)
{
    if (m_core) {
        helix::helix_free(m_core);
        m_core = nullptr;
    }

    m_core = helix::helix_init(configJson.toUtf8().constData());
    return m_core != nullptr;
}

void HelixFfi::connectToServer(const QString &serverId)
{
    if (!m_core) {
        emit errorOccurred();
        return;
    }

    m_errorMessage.clear();
    m_connectionState = HELIX_STATE_CONNECTING;
    emit connectionStateChanged();

    helix::helix_connect(m_core, serverId.toUtf8().constData(),
                        &HelixFfi::stateCallback, this);

    startStatsPolling();
}

void HelixFfi::disconnect()
{
    if (!m_core) return;

    helix::helix_disconnect(m_core);
    stopStatsPolling();
    m_connectionState = HELIX_STATE_DISCONNECTED;
    emit connectionStateChanged();
    emit disconnected();
}

```

```

}

void HelixFfi::refreshStats()
{
    if (!m_core) return;
    helix::helix_poll_stats(m_core, &HelixFfi::statsCallback,
        this);
}

void HelixFfi::stateCallback(int state, const char *info, void
    *userData)
{
    auto *self = static_cast<HelixFfi*>(userData);
    self->updateState(state, QString::fromUtf8(info));
}

void HelixFfi::statsCallback(uint64_t bytesIn, uint64_t bytesOut,
    uint64_t duration, void *userData)
{
    auto *self = static_cast<HelixFfi*>(userData);
    self->updateStats(static_cast<quint64>(bytesIn),
        static_cast<quint64>(bytesOut),
        static_cast<quint64>(duration));
}

void HelixFfi::updateState(int state, const QString &info)
{
    m_connectionState = state;

    if (state == HELIX_STATE_CONNECTED) {
        QJsonDocument doc =
            QJsonDocument::fromJson(info.toUtf8());
        QJsonObject obj = doc.object();
        m_serverName = obj["server_name"].toString();
        m_serverLocation = obj["location"].toString();
        m_ipAddress = obj["tunnel_ip"].toString();
        emit connected(info);
    } else if (state == HELIX_STATE_ERROR) {
        QJsonDocument doc =
            QJsonDocument::fromJson(info.toUtf8());
        QJsonObject obj = doc.object();
        m_errorMessage = obj["error"].toString();
        stopStatsPolling();
    } else if (state == HELIX_STATE_DISCONNECTED) {
        stopStatsPolling();
        emit disconnected();
    }

    emit connectionStateChanged();
    if (state == HELIX_STATE_ERROR) {
        emit errorOccurred();
    }
}

```

```

}

void HelixFfi::updateStats(quint64 bytesIn, quint64 bytesOut,
                           quint64 duration)
{
    m_bytesReceived = bytesIn;
    m_bytesSent = bytesOut;
    m_connectionDuration = duration;
    emit statsUpdated();
}

void HelixFfi::startStatsPolling()
{
    if (!m_statsTimer->isActive()) {
        m_statsTimer->start();
    }
}

void HelixFfi::stopStatsPolling()
{
    m_statsTimer->stop();
}

```

4.5 Qt Project Integration (.pro file)

```

# helix-aurora.pro – qmake project file for Aurora OS
TARGET = ru.auroraos.helixvpn
CONFIG += sailfishapp c++17
PKGCONFIG += dbus-1

QT += quick qml dbus core network

# Rust core library
RUST_DIR = ../helix-core
RUST_TARGET_DIR = $$RUST_DIR/target/release

# Platform detection for Rust target
contains(QT_ARCH, arm) {
    RUST_TARGET = armv7-unknown-linux-gnueabi
} else:contains(QT_ARCH, arm64) {
    RUST_TARGET = aarch64-unknown-linux-gnu
} else {
    RUST_TARGET = x86_64-unknown-linux-gnu
}

# Link against Rust shared library
LIBS += -L$$RUST_TARGET_DIR -lhelix_aurora -ldl -lpthread

# Include generated C headers
INCLUDEPATH += $$PWD/src/ffi

```

```

# Rust library as a build dependency
rust_core.target = $$RUST_TARGET_DIR/libhelix_aurora.so
rust_core.commands = cd $$RUST_DIR && cargo build --release --
target $$RUST_TARGET
QMAKE_EXTRA_TARGETS += rust_core
PRE_TARGETDEPS += $$rust_core.target

# Deploy Rust library with the app
rust_install.files = $$RUST_TARGET_DIR/libhelix_aurora.so
rust_install.path = /usr/share/$$TARGET/lib
INSTALLS += rust_install

# Source files
SOURCES += \
    src/main.cpp \
    src/helixffi.cpp \
    src/connmandbus.cpp \
    src/vpncontroller.cpp \
    src/serverlistmodel.cpp \
    src/settingsmanager.cpp

HEADERS += \
    src/helixffi.h \
    src/helix_aurora.h \
    src/connmandbus.h \
    src/vpncontroller.h \
    src/serverlistmodel.h \
    src/settingsmanager.h

# QML files
qml.files = qml/
qml.path = /usr/share/$$TARGET/qml
INSTALLS += qml

# Desktop file
desktop.files = $$TARGET.desktop
desktop.path = /usr/share/applications
INSTALLS += desktop

# Icon
icon.files = icons/$$TARGET.png
icon.path = /usr/share/icons/hicolor/86x86/apps
INSTALLS += icon

# Sailjail permissions
permissions.files = $$TARGET.permissions
permissions.path = /usr/share/$$TARGET
INSTALLS += permissions

```


4.6 Rust Library Compilation as Shared Object

```
# helix-core/Cargo.toml (workspace root)
[workspace]
members = [
    "crates/helix-core-api",
    "crates/helix-vpn-engine",
    "crates/helix-wireguard",
    "crates/helix-crypto",
    "crates/helix-network",
    "crates/helix-platform-abstraction",
]

[profile.release]
opt-level = "z"
lto = true
codegen-units = 1
panic = "abort"
strip = "symbols"

# crates/helix-core-api/Cargo.toml
[package]
name = "helix-core-api"
version = "0.1.0"
edition = "2021"

[lib]
crate-type = ["cdylib", "staticlib"]
name = "helix_aurora"

[dependencies]
helix-vpn-engine = { path = "../helix-vpn-engine" }
helix-platform-abstraction = { path = "../helix-platform-abstraction" }
tokio = { version = "1", features = ["rt-multi-thread", "sync"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
tracing = "0.1"
```

Build commands for Aurora OS targets:

```
# Install targets
rustup target add aarch64-unknown-linux-gnu
rustup target add armv7-unknown-linux-gnueabihf
rustup target add x86_64-unknown-linux-gnu

# Build for ARM64 (most Aurora devices)
cargo build --release --target aarch64-unknown-linux-gnu

# Build for ARMv7 (older devices)
cargo build --release --target armv7-unknown-linux-gnueabihf
```

```
# Build for emulator (x86_64)
cargo build --release --target x86_64-unknown-linux-gnu
```

5. UI Implementation

5.1 Application Entry Point

```
// main.cpp
#include <sailfishapp.h>
#include <QScopedPointer>
#include <QQuickView>
#include <QMLContext>
#include <QGuiApplication>
#include "vpncontroller.h"
#include "serverlistmodel.h"
#include "settingsmanager.h"
#include "helixffi.h"

int main(int argc, char *argv[])
{
    QScopedPointer<QGuiApplication>
        app(SailfishApp::application(argc, argv));
    QScopedPointer<QQuickView> view(SailfishApp::createView());

    // Register C++ types with QML
    qmlRegisterType<HelixFfi>("Helix.Vpn", 1, 0, "HelixCore");
    qmlRegisterType<VpnController>("Helix.Vpn", 1, 0,
        "VpnController");
    qmlRegisterType<ServerListModel>("Helix.Vpn", 1, 0,
        "ServerListModel");
    qmlRegisterType<SettingsManager>("Helix.Vpn", 1, 0,
        "SettingsManager");

    // Create singleton instances
    HelixFfi helixCore;
    VpnController vpnController(&helixCore);
    ServerListModel serverModel;
    SettingsManager settings;

    // Initialize the Rust core
    QString configJson = settings.coreConfigurationJson();
    helixCore.initialize(configJson);

    // Expose to QML
    view->rootContext()->setContextProperty("vpnController",
        &vpnController);
    view->rootContext()->setContextProperty("serverModel",
        &serverModel);
    view->rootContext()->setContextProperty("appSettings",
        &settings);
```

```

view->rootContext()->setContextProperty("helixCore",
    &helixCore);

view->setSource(SailfishApp::pathTo("qml/main.qml"));
view->show();

return app->exec();
}

```

5.2 Main QML Application File

```

// qml/main.qml
import QtQuick 2.6
import Sailfish.Silica 1.0
import "pages"
import "cover"

ApplicationWindow {
    id: appWindow

    // Initial page shown on launch
    initialPage: Component { MainPage { } }

    // Cover page (active frame shown on home screen)
    cover: Qt.resolvedUrl("cover/CoverPage.qml")

    // Handle application state changes
    onApplicationActiveChanged: {
        if (applicationActive) {
            vpnController.refreshStatus();
        }
    }

    // Global connection for state changes
    Connections {
        target: vpnController
        onShowError: {
            // Error handling will be shown on current page
        }
    }
}

```

5.3 Main Page (Connection UI)

```

// qml/pages/MainPage.qml
import QtQuick 2.6
import Sailfish.Silica 1.0
import Helix.Vpn 1.0

Page {
    id: mainPage

```

```

property bool isConnected: vpnController.connectionState ===
    VpnController.Connecting
property bool isConnected: vpnController.connectionState ===
    VpnController.Connected
property bool isDisconnected: vpnController.connectionState
    === VpnController.Disconnected

// Pull-down menu for quick actions
PullDownMenu {
    MenuItem {
        text: qsTr("Settings")
        onClicked:
            pageStack.push(Qt.resolvedUrl("SettingsPage.qml"))
    }
    MenuItem {
        text: qsTr("Server List")
        onClicked:
            pageStack.push(Qt.resolvedUrl("ServerListPage.qml"))
    }
    MenuItem {
        text: qsTr("Refresh Servers")
        onClicked: serverModel.refresh()
    }
}

SilicaFlickable {
    anchors.fill: parent
    contentHeight: column.height + Theme.paddingLarge

    Column {
        id: column
        width: parent.width
        spacing: Theme.paddingLarge

        // Page header
        PageHeader {
            title: qsTr("Helix VPN")
            description: isConnected ? qsTr("Protected") :
                qsTr("Not Connected")
        }

        // Connection status card
        Item {
            width: parent.width
            height: statusColumn.height + Theme.paddingLarge
            * 2

            Rectangle {
                anchors.fill: parent
                anchors.margins: Theme.horizontalPageMargin
                radius: Theme.paddingMedium
            }
        }
    }
}

```

```

        color: isConnected ? "#1B5E20" :
(isConnecting ? "#E65100" : "#37474F")
        opacity: 0.3
    }

    Column {
        id: statusColumn
        anchors.centerIn: parent
        width: parent.width -
Theme.horizontalPageMargin * 4
        spacing: Theme.paddingSmall

        // Shield icon (using Unicode)
        Label {
            anchors.horizontalCenter:
parent.horizontalCenter
            text: isConnected ? "\uE32A" : "\uE32B"
            font.pixelSize: Theme.fontSizeHuge * 2
            color: isConnected ? Theme.primaryColor :
Theme.secondaryColor
        }

        // Status text
        Label {
            anchors.horizontalCenter:
parent.horizontalCenter
            text: isConnected ? qsTr("Connected")
                : isConnecting ?
qsTr("Connecting...")
                : qsTr("Not Connected")
            font.pixelSize: Theme.fontSizeExtraLarge
            color: isConnected ?
Theme.highlightColor : Theme.primaryColor
        }

        // Server info (when connected)
        Label {
            anchors.horizontalCenter:
parent.horizontalCenter
            text: vpnController.serverName + " - " +
vpnController.serverLocation
            visible: isConnected || isConnecting
            font.pixelSize: Theme.fontSizeMedium
            color: Theme.secondaryColor
            truncationMode: TruncationMode.Fade
        }

        // IP address (when connected)
        Label {
            anchors.horizontalCenter:
parent.horizontalCenter
            text: vpnController.ipAddress
            visible: isConnected

```

```

        font.pixelSize: Theme.fontSizeSmall
        color: Theme.secondaryHighlightColor
    }
}

// Connection button
Button {
    id: connectButton
    anchors.horizontalCenter: parent.horizontalCenter
    preferredWidth: Theme.buttonWidthLarge
    text: isConnected ? qsTr("Disconnect")
        : isConnecting ? qsTr("Cancel")
        : qsTr("Connect")

    onClicked: {
        if (isConnected) {
            vpnController.disconnect();
        } else if (isConnecting) {
            vpnController.cancelConnection();
        } else {
            vpnController.connectToBestServer();
        }
    }
}

// Quick server selection (disconnected state)
ValueButton {
    anchors.horizontalCenter: parent.horizontalCenter
    label: qsTr("Server")
    value: serverModel.selectedServerName ||
qsTr("Auto")
    visible: isDisconnected
    onClicked:
pageStack.push(Qt.resolvedUrl("ServerListPage.qml"))
}

// Statistics (connected state)
Column {
    anchors.horizontalCenter: parent.horizontalCenter
    visible: isConnected
    width: parent.width - Theme.horizontalPageMargin
    spacing: Theme.paddingSmall

    * 2

    SectionHeader {
        text: qsTr("Statistics")
    }

    // Download speed
    DetailItem {
        label: qsTr("Received")
    }
}

```

```

        value:
Format.formatFileSize(vpnController.bytesReceived) + "B"
    }

    // Upload speed
    DetailItem {
        label: qsTr("Sent")
        value:
Format.formatFileSize(vpnController.bytesSent) + "B"
    }

    // Duration
    DetailItem {
        label: qsTr("Duration")
        value: {
            var mins =
Math.floor(vpnController.connectionDuration / 60);
            var secs =
vpnController.connectionDuration % 60;
            return mins + ":" + (secs < 10 ? "0" :
"") + secs;
        }
    }

    // Protocol
    DetailItem {
        label: qsTr("Protocol")
        value: "WireGuard"
    }
}

// Last error (if any)
Label {
    anchors.horizontalCenter: parent.horizontalCenter
    text: vpnController.lastError
    visible: text.length > 0 && isDisconnected
    color: Theme.errorColor
    font.pixelSize: Theme.fontSizeSmall
    wrapMode: Text.WordWrap
    width: parent.width - Theme.horizontalPageMargin
    * 2
    horizontalAlignment: Text.AlignHCenter
}
}

VerticalScrollDecorator {}
}
}

```

5.4 Cover Page (Active Frame)

```
// qml/cover/CoverPage.qml
import QtQuick 2.6
import Sailfish.Silica 1.0

CoverBackground {
    id: cover

    property bool isConnected: vpnController.connectionState ===
        VpnController.Connected
    property bool isConnecting: vpnController.connectionState ===
        VpnController.Connecting

    // Status icon
    Label {
        id: statusIcon
        anchors.horizontalCenter: parent.horizontalCenter
        anchors.top: parent.top
        anchors.topMargin: Theme.paddingLarge
        text: isConnected ? "\uE32A" : "\uE32B"
        font.pixelSize: Theme.fontSizeHuge * 1.5
        color: isConnected ? Theme.highlightColor :
            Theme.secondaryColor
    }

    // Status text
    Label {
        id: statusLabel
        anchors.horizontalCenter: parent.horizontalCenter
        anchors.top: statusIcon.bottom
        anchors.topMargin: Theme.paddingSmall
        text: isConnected ? qsTr("Connected")
            : isConnecting ? qsTr("Connecting...")
            : qsTr("Helix VPN")
        font.pixelSize: Theme.fontSizeMedium
        color: Theme.primaryColor
    }

    // Server info (when connected)
    Label {
        anchors.horizontalCenter: parent.horizontalCenter
        anchors.top: statusLabel.bottom
        anchors.topMargin: Theme.paddingSmall
        text: vpnController.serverName
        visible: isConnected
        font.pixelSize: Theme.fontSizeSmall
        color: Theme.secondaryHighlightColor
        truncationMode: TruncationMode.Fade
        width: parent.width - Theme.paddingMedium * 2
        horizontalAlignment: Text.AlignHCenter
    }
}
```



```

// Cover actions (max 2)
CoverActionList {
    id: coverActionList

    // Connect/Disconnect action
    CoverAction {
        iconSource: isConnected
            ? "image://theme/icon-cover-cancel"
            : "image://theme/icon-cover-new"
        onTriggered: {
            if (isConnected) {
                vpnController.disconnect();
            } else if (!isConnecting) {
                vpnController.connectToBestServer();
            }
        }
    }

    // Switch server (when connected)
    CoverAction {
        iconSource: "image://theme/icon-cover-next"
        visible: isConnected
        onTriggered: {
            // Activate app and show server list
            appWindow.activate();
            pageStack.push(Qt.resolvedUrl("../pages/
ServerListPage.qml"));
        }
    }
}

// Only animate when cover is active
onStatusChanged: {
    if (status === Cover.Active) {
        vpnController.refreshStatus();
    }
}
}

```

5.5 Server List Page

```

// qml/pages/ServerListPage.qml
import QtQuick 2.6
import Sailfish.Silica 1.0

Page {
    id: serverListPage

    // Pull-down menu
    PullDownMenu {

```

```

MenuItem {
    text: qsTr("Refresh")
    onClicked: serverModel.refresh()
}
MenuItem {
    text: qsTr("Sort by Latency")
    onClicked: serverModel.sortByLatency()
}
}

SilicaListView {
    id: serverList
    anchors.fill: parent
    model: serverModel
    clip: true

    // Header
    header: PageHeader {
        title: qsTr("Select Server")
    }

    // Empty state
    ViewPlaceholder {
        enabled: serverList.count === 0
        text: qsTr("No servers available")
        hintText: qsTr("Pull down to refresh")
    }

    // Delegate for each server
    delegate: ListItem {
        id: serverItem
        contentHeight: Theme.itemSizeMedium

        property bool isCurrentServer: model.serverId ===
serverModel.selectedServerId
        property int latency: model.latencyMs

        // Selection indicator
        Rectangle {
            anchors.left: parent.left
            anchors.top: parent.top
            anchors.bottom: parent.bottom
            width: Theme.paddingSmall
            color: isCurrentServer ? Theme.highlightColor :
"transparent"
        }

        Row {
            anchors.fill: parent
            anchors.leftMargin: Theme.horizontalPageMargin +
Theme.paddingSmall
            anchors.rightMargin: Theme.horizontalPageMargin

```

```

        spacing: Theme.paddingMedium

        // Country flag (emoji or icon)
        Label {
            anchors.verticalCenter: parent.verticalCenter
            text: model.countryFlag
            font.pixelSize: Theme.fontSizeLarge
        }

        // Server info column
        Column {
            anchors.verticalCenter: parent.verticalCenter
            width: parent.width - parent.spacing * 2 -
                flagLabel.width - latencyLabel.width

            Label {
                text: model.city + ", " + model.country
                font.pixelSize: Theme.fontSizeMedium
                color: serverItem.highlighted ?
                    Theme.highlightColor : Theme.primaryColor
                truncationMode: TruncationMode.Fade
                width: parent.width
            }

            Label {
                text: model.serverName
                font.pixelSize: Theme.fontSizeSmall
                color: Theme.secondaryColor
                truncationMode: TruncationMode.Fade
                width: parent.width
            }
        }

        // Latency indicator
        Label {
            id: latencyLabel
            anchors.verticalCenter: parent.verticalCenter
            text: latency > 0 ? latency + " ms" : "-"
            font.pixelSize: Theme.fontSizeSmall
            color: latency < 100 ? "#4CAF50"
                : latency < 200 ? "#FF9800"
                : latency > 0 ? "#F44336"
                : Theme.secondaryColor
        }
    }

    // Menu for this server
    menu: ContextMenu {
        MenuItem {
            text: qsTr("Connect")
            onClicked: {

```

```

        serverModel.selectedServerId =
model.serverId;

vpnController.connectToServer(model.serverId);
        pageStack.pop();
    }
}
MenuItem {
    text: qsTr("Test Latency")
    onClicked:
serverModel.testLatency(model.index)
}
MenuItem {
    text: qsTr("Add to Favorites")
    visible: !model.isFavorite
    onClicked:
serverModel.addFavorite(model.serverId)
}
MenuItem {
    text: qsTr("Remove from Favorites")
    visible: model.isFavorite
    onClicked:
serverModel.removeFavorite(model.serverId)
}
}

// Tap to select and connect
onClicked: {
    serverModel.selectedServerId = model.serverId;
    vpnController.connectToServer(model.serverId);
    pageStack.pop();
}
}

VerticalScrollDecorator {}
}
}

```

5.6 Settings Page

```

// qml/pages/SettingsPage.qml
import QtQuick 2.6
import Sailfish.Silica 1.0

Page {
    id: settingsPage

    SilicaFlickable {
        anchors.fill: parent
        contentHeight: settingsColumn.height + Theme.paddingLarge

        Column {

```

```

        id: settingsColumn
        width: parent.width
        spacing: 0

        PageHeader {
            title: qsTr("Settings")
        }

        // Connection Settings Section
        SectionHeader {
            text: qsTr("Connection")
        }

        // Auto-connect
        TextSwitch {
            text: qsTr("Auto-connect on launch")
            checked: appSettings.autoConnect
            onCheckedChanged: appSettings.autoConnect =
checked
            description: qsTr("Automatically connect when app
starts")
        }

        // Kill switch
        TextSwitch {
            text: qsTr("Kill Switch")
            checked: appSettings.killSwitchEnabled
            onCheckedChanged: appSettings.killSwitchEnabled =
checked
            description: qsTr("Block internet when VPN
disconnects")
        }

        // Split tunneling
        ValueButton {
            label: qsTr("Split Tunneling")
            value: appSettings.splitTunnelEnabled ?
qsTr("Enabled") : qsTr("Disabled")
            onClicked:
pageStack.push(Qt.resolvedUrl("SplitTunnelPage.qml"))
        }

        // Protocol selection
        ValueButton {
            label: qsTr("Protocol")
            value: appSettings.protocol
            onClicked: {
                var dialog =
pageStack.push("Sailfish.Silica.TimePickerDialog", {
                    // Use a custom dialog for protocol
selection
                });
            }
        }

```

```

    }

    Divider { }

    // Privacy Settings Section
    SectionHeader {
        text: qsTr("Privacy")
    }

    // DNS leak protection
    TextSwitch {
        text: qsTr("DNS Leak Protection")
        checked: appSettings.dnsLeakProtection
        onCheckedChanged: appSettings.dnsLeakProtection =
checked
    }

    // IPv6 leak protection
    TextSwitch {
        text: qsTr("IPv6 Leak Protection")
        checked: appSettings.ipv6LeakProtection
        onCheckedChanged: appSettings.ipv6LeakProtection
= checked
        description: qsTr("Disable IPv6 traffic outside
VPN")
    }

    Divider { }

    // App Settings Section
    SectionHeader {
        text: qsTr("Application")
    }

    // Show in cover
    TextSwitch {
        text: qsTr("Show connection in cover")
        checked: appSettings.coverEnabled
        onCheckedChanged: appSettings.coverEnabled =
checked
    }

    // Notifications
    TextSwitch {
        text: qsTr("Connection notifications")
        checked: appSettings.notificationsEnabled
        onCheckedChanged:
appSettings.notificationsEnabled = checked
    }

    // Version info
    Label {
        anchors.horizontalCenter: parent.horizontalCenter

```

```

        text: "Helix VPN v" + appSettings.appVersion
        font.pixelSize: Theme.fontSizeSmall
        color: Theme.secondaryColor
    }

    // About button
    Button {
        anchors.horizontalCenter: parent.horizontalCenter
        preferredWidth: Theme.buttonWidthMedium
        text: qsTr("About")
        onClicked:
            pageStack.push(Qt.resolvedUrl("AboutPage.qml"))
    }
}

VerticalScrollDecorator {}
}
}

```

5.7 VpnController (QML-Exposed Controller)

```

// vpncontroller.h
#ifndef VPNCONTROLLER_H
#define VPNCONTROLLER_H

#include <QObject>
#include "helixffi.h"

class VpnController : public QObject
{
    Q_OBJECT
    Q_PROPERTY(int connectionState READ connectionState NOTIFY
        connectionStateChanged)
    Q_PROPERTY(QString serverName READ serverName NOTIFY
        connectionInfoChanged)
    Q_PROPERTY(QString serverLocation READ serverLocation NOTIFY
        connectionInfoChanged)
    Q_PROPERTY(QString ipAddress READ ipAddress NOTIFY
        connectionInfoChanged)
    Q_PROPERTY(quint64 bytesReceived READ bytesReceived NOTIFY
        statsUpdated)
    Q_PROPERTY(quint64 bytesSent READ bytesSent NOTIFY
        statsUpdated)
    Q_PROPERTY(quint64 connectionDuration READ connectionDuration
        NOTIFY statsUpdated)
    Q_PROPERTY(QString lastError READ lastError NOTIFY
        errorOccurred)

public:
    enum ConnectionState {
        Disconnected = 0,
        Connecting = 1,
        Connected = 2,
    }

```

```

        Disconnecting = 3,
        Error = 4,
        Reconnecting = 5
};
Q_ENUM(ConnectionState)

explicit VpnController(HelixFfi *core, QObject *parent =
    nullptr);

int connectionState() const;
QString serverName() const;
QString serverLocation() const;
QString ipAddress() const;
quint64 bytesReceived() const;
quint64 bytesSent() const;
quint64 connectionDuration() const;
QString lastError() const;

Q_INVOKABLE void connectToServer(const QString &serverId);
Q_INVOKABLE void connectToBestServer();
Q_INVOKABLE void disconnect();
Q_INVOKABLE void cancelConnection();
Q_INVOKABLE void refreshStatus();

signals:
    void connectionStateChanged();
    void connectionInfoChanged();
    void statsUpdated();
    void errorOccurred();
    void connected(const QString &serverInfo);
    void disconnected();
    void showError(const QString &message);

private slots:
    void onHelixStateChanged();
    void onHelixStatsUpdated();
    void onHelixError();

private:
    HelixFfi *m_helixCore;
    int m_state = Disconnected;
    QString m_lastError;
};

#endif // VPNCONTROLLER_H

```

6. Platform-Specific Features

6.1 Ambiance / Theme Integration

Aurora OS uses "ambiances" (themes) that define the color palette. All UI components must use Theme properties:

```
// Correct – uses Theme
Label {
    color: Theme.primaryColor
    font.pixelSize: Theme.fontSizeMedium
}
```

```
// Incorrect – hardcoded colors
Label {
    color: "#FFFFFF"
    font.pixelSize: 16
}
```

Key Theme properties for VPN UI:

- Theme.primaryColor — Main text, interactive elements
- Theme.secondaryColor — Descriptive text, non-interactive
- Theme.highlightColor — Pressed/selected items
- Theme.secondaryHighlightColor — Accent for secondary info
- Theme.errorColor — Error states
- Theme.itemSizeSmall/Medium/Large — Row heights
- Theme.paddingSmall/Medium/Large — Spacing
- Theme.horizontalPageMargin — Side margins

6.2 Sailfish OS Notifications

```
// notificationsservice.h
#ifndef NOTIFICATIONSERVICE_H
#define NOTIFICATIONSERVICE_H

#include <QObject>
#include <nemonotifications.h>

class NotificationService : public QObject
{
    Q_OBJECT
public:
    explicit NotificationService(QObject *parent = nullptr);

    Q_INVOKABLE void notifyConnected(const QString &server);
    Q_INVOKABLE void notifyDisconnected();
    Q_INVOKABLE void notifyError(const QString &message);
    Q_INVOKABLE void notifyKillSwitchTriggered();
};
```

```

private:
    void sendNotification(const QString &summary, const QString
                        &body,
                        const QString &icon);
};

#endif

// notificationsservice.cpp
#include "notificationsservice.h"
#include <QDebug>

NotificationService::NotificationService(QObject *parent) :
    QObject(parent) {}

void NotificationService::sendNotification(const QString
    &summary,
    const QString &body,
    const QString &icon)
{
    Notification notification;
    notification.setAppName("Helix VPN");
    notification.setSummary(summary);
    notification.setBody(body);
    notification.setIcon(icon);
    notification.setPreviewSummary(summary);
    notification.setPreviewBody(body);
    notification.publish();
}

void NotificationService::notifyConnected(const QString &server)
{
    sendNotification(
        QObject::tr("VPN Connected"),
        QObject::tr("Connected to %1").arg(server),
        "icon-lock-enabled"
    );
}

void NotificationService::notifyDisconnected()
{
    sendNotification(
        QObject::tr("VPN Disconnected"),
        QObject::tr("Your VPN connection has ended"),
        "icon-lock-disabled"
    );
}

void NotificationService::notifyError(const QString &message)
{
    sendNotification(
        QObject::tr("VPN Error"),

```

```

        message,
        "icon-lock-error"
    );
}

void NotificationService::notifyKillSwitchTriggered()
{
    sendNotification(
        QObject::tr("Kill Switch Active"),
        QObject::tr("Internet blocked – VPN connection lost"),
        "icon-lock-warning"
    );
}

```

6.3 Background Execution Model

Aurora OS uses Sailjail sandboxing. For background operation:

```

# ru.auroraos.helixvpn.desktop
[Desktop Entry]
Type=Application
Name=Helix VPN
Icon=ru.auroraos.helixvpn
Exec=/usr/bin/ru.auroraos.helixvpn
X-Nemo-Application-Type=silica-qt5
X-Nemo-Single-Instance=no

# Sailjail permissions
X-Sailjail-
    Permissions=Network;Internet;VPN;Notifications;AppLaunch
X-Sailjail-OrganizationName=ru.auroraos
X-Sailjail-ApplicationName=helixvpn

# ru.auroraos.helixvpn.permissions (additional permissions file)
# Allow D-Bus access to ConnMan
[Session Bus]
own=ru.auroraos.helixvpn.*
talk=net.connman.vpn

[System Bus]
talk=net.connman.vpn
talk=net.connman

```

Background execution considerations:

- **Covers:** App runs as a Cover when minimized — keep minimal resources
- **No true background:** Apps may be suspended; use ConnMan for persistent VPN
- **Keep-alive:** ConnMan itself maintains the VPN connection, not the app

- **Reconnect:** App reconnects to D-Bus on activation to check status

6.4 Power Management

```
// powermanager.h – CPU wake lock and power management
#ifndef POWERMANAGER_H
#define POWERMANAGER_H

#include <QObject>
#include <keepalive backgroundactivity.h>

class PowerManager : public QObject
{
    Q_OBJECT
public:
    explicit PowerManager(QObject *parent = nullptr);
    ~PowerManager();

    Q_INVOKABLE void requestKeepAlive();
    Q_INVOKABLE void releaseKeepAlive();

private:
    BackgroundActivity *m_activity = nullptr;
};

#endif
```

Power management strategies:

- Rust core uses async I/O (tokio) to minimize CPU wakeups
- Stats polling from QML uses 2-second intervals (not continuous)
- Cover page pauses animations when inactive
- App releases resources when backgrounded
- ConnMan (system service) handles actual tunnel I/O, not the app

6.5 Harbor (OpenRepos) Distribution

Distribution options for Aurora OS:

Channel	Type	Requirements	Audience
OpenRepos	Community	Basic validation	Power users, early adopters
Harbor	Official	Strict validation	General users
Chum	Developer	Build from source	Developers
Manual	Direct	None	

Channel	Type	Requirements	Audience
			Testing, enterprise

Harbor validation requirements:

- Must use Sailfish Silica components (no Qt Quick Controls)
- Must have proper .desktop file with Sailjail permissions
- Must include icon in all required sizes
- Must not use private APIs
- Must handle all orientations
- Must have active cover
- RPM must be signed

7. Build System

7.1 Sailfish SDK / Aurora SDK Setup

The Aurora SDK is based on the Sailfish SDK with Aurora-specific targets.

```
# 1. Download Aurora SDK from https://developer.auroraos.ru
# 2. Install (includes Qt Creator, build engines, emulator)
```

```
# 3. Verify SDK installation
sfdk --version
```

```
# 4. List available build targets
sfdk targets list
```

```
# 5. List available devices (emulator + hardware)
sfdk device list
```

7.2 sfdk CLI Build Process

```
# Step 1: Select build target (emulator = x86_64)
sfdk config target=AuroraOS-5.1.2.10-base-x86_64
```

```
# Step 2: Create build directory
mkdir -p helix-aurora.build.x86_64 && cd helix-aurora.build.x86_64
```

```
# Step 3: Build the project
sfdk build ../helix-aurora
```

```
# For ARM devices:
sfdk config target=AuroraOS-5.1.2.10-base-armv7hl
mkdir -p helix-aurora.build.armv7hl && cd helix-aurora.build.armv7hl
```

```
sfdk build ../helix-aurora
```

```
# For ARM64 devices:
```

```
sfdk config target=AuroraOS-5.1.2.10-base-aarch64
```

```
mkdir -p helix-aurora.build.aarch64 && cd helix-  
aurora.build.aarch64
```

```
sfdk build ../helix-aurora
```

7.3 CMake Alternative

```
# CMakeLists.txt for Aurora OS
```

```
project(helix-aurora LANGUAGES CXX)
```

```
cmake_minimum_required(VERSION 3.16)
```

```
set(CMAKE_CXX_STANDARD 17)
```

```
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

```
set(CMAKE_AUTOMOC ON)
```

```
set(CMAKE_AUTORCC ON)
```

```
find_package(Qt6 REQUIRED COMPONENTS Core Quick DBus Network)
```

```
find_package(PkgConfig REQUIRED)
```

```
pkg_check_modules(DBUS REQUIRED dbus-1)
```

```
# Rust core
```

```
set(RUST_DIR "${CMAKE_SOURCE_DIR}/../helix-core")
```

```
set(RUST_TARGET "aarch64-unknown-linux-gnu")
```

```
set(RUST_LIB_DIR "${RUST_DIR}/target/${RUST_TARGET}/release")
```

```
add_custom_command(
```

```
    OUTPUT ${RUST_LIB_DIR}/libhelix_aurora.so
```

```
    COMMAND cargo build --release --target ${RUST_TARGET}
```

```
    WORKING_DIRECTORY ${RUST_DIR}
```

```
    COMMENT "Building Rust core..."
```

```
)
```

```
add_custom_target(rust_core ALL DEPENDS ${RUST_LIB_DIR}/
```

```
libhelix_aurora.so)
```

```
# Sources
```

```
set(SOURCES
```

```
    src/main.cpp
```

```
    src/helixffi.cpp
```

```
    src/connmandbus.cpp
```

```
    src/vpncontroller.cpp
```

```
    src/serverlistmodel.cpp
```

```
    src/settingsmanager.cpp
```

```
    src/notificationservice.cpp
```

```
)
```

```
set(HEADERS
```

```

    src/helixffi.h
    src/helix_aurora.h
    src/connmandbus.h
    src/vpncontroller.h
    src/serverlistmodel.h
    src/settingsmanager.h
    src/notificationservice.h
)

add_executable(${PROJECT_NAME} ${SOURCES} ${HEADERS})
add_dependencies(${PROJECT_NAME} rust_core)

target_include_directories(${PROJECT_NAME} PRIVATE
    src/ffi
    ${DBUS_INCLUDE_DIRS}
)

target_link_libraries(${PROJECT_NAME} PRIVATE
    Qt6::Core
    Qt6::Quick
    Qt6::DBus
    Qt6::Network
    ${RUST_LIB_DIR}/libhelix_aurora.so
    ${DBUS_LIBRARIES}
    dl
    pthread
)

# Install
install(TARGETS ${PROJECT_NAME} DESTINATION /usr/bin)
install(DIRECTORY qml DESTINATION /usr/share/${PROJECT_NAME})
install(FILES ${RUST_LIB_DIR}/libhelix_aurora.so DESTINATION /usr/
    share/${PROJECT_NAME}/lib)

```

7.4 RPM Spec File

```

# rpm/ru.auroraos.helixvpn.spec
Name:          ru.auroraos.helixvpn
Summary:       Helix VPN client for Aurora OS
Version:       0.1.0
Release:       1
License:       GPLv3
URL:           https://helixvpn.com
Source0:       %{name}-%{version}.tar.gz
BuildArch:     %{_arch}

BuildRequires: pkgconfig(Qt6Core)
BuildRequires: pkgconfig(Qt6Quick)
BuildRequires: pkgconfig(Qt6DBus)
BuildRequires: pkgconfig(Qt6Network)
BuildRequires: pkgconfig(dbus-1)

```

```
BuildRequires: sailfishsilica-qt6-devel >= 0.10.9
BuildRequires: libsailfishapp-devel
BuildRequires: rust >= 1.75
BuildRequires: cargo
BuildRequires: cbindgen
BuildRequires: pkgconfig(keepalive)
```

```
Requires: sailfishsilica-qt6 >= 0.10.9
Requires: libsailfishapp
Requires: connman >= 1.41
Requires: connman-vpn-plugin-wireguard
Requires: nemo-qml-plugin-notifications-qt6
Requires: nemo-qml-plugin-dbus-qt6
```

%description

Helix VPN – Secure, fast WireGuard VPN client for Aurora OS.
Features automatic server selection, kill switch, split tunneling,
and full ConnMan integration.

%prep

%setup -q

Build Rust core

cd ../helix-core

cargo build --release --target %{_rust_target}

cbindgen --crate helix-core-api --output src/ffi/helix_aurora.h

%build

%qmake5 CONFIG+=release

make %{?_smp_mflags}

%install

%qmake5_install

Install Rust shared library

mkdir -p %{buildroot}%{_datadir}/%{name}/lib

cp ../helix-core/target/%{_rust_target}/release/libhelix_aurora.so
\

 %{buildroot}%{_datadir}/%{name}/lib/

Desktop file

cp %{name}.desktop %{buildroot}%{_datadir}/applications/

Icon

mkdir -p %{buildroot}%{_datadir}/icons/hicolor/86x86/apps

cp icons/86x86/%{name}.png %{buildroot}%{_datadir}/icons/hicolor/
86x86/apps/

mkdir -p %{buildroot}%{_datadir}/icons/hicolor/108x108/apps

cp icons/108x108/%{name}.png %{buildroot}%{_datadir}/icons/
hicolor/108x108/apps/

mkdir -p %{buildroot}%{_datadir}/icons/hicolor/128x128/apps

cp icons/128x128/%{name}.png %{buildroot}%{_datadir}/icons/


```

hicolor/128x128/apps/
mkdir -p ${buildroot}/${_datadir}/icons/hicolor/172x172/apps
cp icons/172x172/${name}.png ${buildroot}/${_datadir}/icons/
hicolor/172x172/apps/

# Permissions file
mkdir -p ${buildroot}/${_datadir}/${name}
cp ${name}.permissions ${buildroot}/${_datadir}/${name}/

%files
%defattr(-,root,root,-)
%{_bindir}/${name}
%{_datadir}/${name}/qml/
%{_datadir}/${name}/lib/libhelix_aurora.so
%{_datadir}/applications/${name}.desktop
%{_datadir}/icons/hicolor/*/apps/${name}.png
%{_datadir}/${name}/${name}.permissions

```

7.5 Target Architectures

Architecture	Target Devices	sfdk Target	Rust Target
x86_64	Emulator only	Aurora0S-*-base-x86_64	x86_64-unknown-linux-gnu
armv7hl	Older devices (F+ R570E)	Aurora0S-*-base-armv7hl	armv7-unknown-linux-gnueabi
aarch64	Modern devices	Aurora0S-*-base-aarch64	aarch64-unknown-linux-gnu

7.6 Emulator Testing

```

# Build for emulator
sfdk config target=Aurora0S-5.1.2.10-base-x86_64
sfdk build

# Deploy to emulator
sfdk deploy --sdk

# Run on emulator
sfdk device exec /usr/bin/ru.auroraos.helixvpn

# Debug on emulator
sfdk debug /usr/bin/ru.auroraos.helixvpn

# Check logs
sfdk device exec journalctl --user -u ru.auroraos.helixvpn -f

```

8. Comparison with Other Platforms

8.1 What Makes Aurora OS Different

Aspect	Aurora OS	Android	iOS	Linux Desktop
UI Framework	Qt6/QML + Silica	Jetpack Compose / XML	SwiftUI	GTK4 / Qt6
VPN API	ConnMan D-Bus	VpnService	NEPacketTunnelProvider	TUN + netlink
Background	Cover-based	Foreground Service	Network Extension	systemd service
Sandbox	Sailjail (Firejail)	Android sandbox	App Sandbox	None (flatpak)
Distribution	RPM / OpenRepos	APK / Play Store	IPA / App Store	DEB/ RPM/ Flatpak
Market	Russian niche	Global	Global	Global
Devices	Mid-range ARM	All ranges	Premium only	x86/ ARM

8.2 Code Reuse from Rust Core (~75%)

Component	Code Lines	Aurora Reuse
helix-wireguard	~8,000	100%
helix-vpn-engine	~6,000	100%
helix-crypto	~2,000	100%
helix-network	~3,000	100%
helix-core-api	~4,000	100% (FFI additions: ~300 lines)
Platform Adapter	~2,500	80% (Linux adapter with ConnMan D-Bus)
C++ Bridge	~800	Aurora-specific
QML UI	~1,500	Aurora-specific
Total Reuse		~75%

8.3 Unique UI Patterns

Aurora OS has distinctive UI patterns not found on other platforms:

1. **Pulley Menus:** Pull down from top to reveal menu items (no hamburger menu)
2. **Active Covers:** App representation on home screen with actions (no system tray)
3. **Swipe Navigation:** Back gesture from screen edge (no back button)
4. **Remorse Actions:** Undo pattern for destructive actions
5. **Ambience-Themed:** UI colors adapt to OS theme automatically
6. **Peek:** Peek at previous screen by dragging from edge

8.4 Platform-Specific Adaptations

Feature	Adaptation Required
Connection status	Cover page instead of system tray
Quick actions	Cover actions (max 2) instead of notification buttons
Settings integration	In-app only (no system settings plugin)
Background VPN	ConnMan maintains connection; app shows status
Notifications	nemo-qml-plugin-notifications
Sandboxing	Declare D-Bus permissions in .desktop file
Icon sizes	86x86, 108x108, 128x128, 172x172 PNG
App lifecycle	Minimize to cover; may be suspended

9. Implementation Phases

9.1 Phase 1: Rust Core FFI Bridge and ConnMan Plugin (Weeks 1-2)

Goals:

- Compile helix-core for aarch64-unknown-linux-gnu
- Implement FFI exports (helix_aurora.rs)
- Generate C headers via cbindgen
- Build libhelix_aurora.so as shared library
- Implement ConnMan D-Bus client in C++
- Test ConnMan VPN control via D-Bus

Deliverables:

```
helix-aurora/
├── src/
│   ├── helixffi.cpp/h          # C++ wrapper for Rust FFI
│   ├── connmandbus.cpp/h      # ConnMan D-Bus client
│   └── helix_aurora.h          # Generated C header
├── ../helix-core/
│   └── crates/
│       └── helix-core-api/
│           └── src/
│               └── ffi_aurora.rs # Rust FFI exports
```

Verification:

Test Rust core loads

```
cargo test --release --target aarch64-unknown-linux-gnu
```

Test C++ bridge

```
sfdk build && sfdk deploy --sdk && sfdk device exec test_ffi
```

Test ConnMan D-Bus communication

```
sfdk device exec /usr/bin/ru.auroraos.helixvpn --test-dbus
```

9.2 Phase 2: Basic UI (Weeks 3-4)

Goals:

- Main page with connect/disconnect
- Server selection page with latency
- Cover page with status and actions
- Settings page skeleton
- Connection status display
- Pull-down menus

Deliverables:

```
qml/
├── main.qml                    # ApplicationWindow
├── pages/
│   ├── MainPage.qml           # Connection UI
│   ├── ServerListPage.qml     # Server selection
│   └── SettingsPage.qml       # Basic settings
└── cover/
    └── CoverPage.qml          # Active frame
```

UI Screens: | Screen | Components | |-----|-----| | Main |
Status card, connect button, stats, server selector | | Server List |
SilicaListView, latency badges, country flags | | Settings |
TextSwitch, ValueButton, section headers | | Cover | Status icon,
connect/disconnect action |

9.3 Phase 3: Advanced Features (Weeks 5-6)

Goals:

- Split tunneling UI
- Kill switch toggle with ConnMan integration
- DNS leak protection
- IPv6 leak protection
- Server favorites
- Auto-connect on launch
- Connection notifications
- Statistics persistence

Deliverables:

```
qml/pages/  
├─ SplitTunnelPage.qml      # App-based split tunneling  
├─ ConnectionLogPage.qml    # Connection history  
└─ AboutPage.qml           # App info, licenses  
  
src/  
├─ notificationsservice.cpp/h # Sailfish notifications  
├─ settingsmanager.cpp/h     # QSettings persistence  
└─ serverlistmodel.cpp/h     # Server data model
```

9.4 Phase 4: Polish and Distribution (Weeks 7-8)

Goals:

- Ambiance/theme compliance testing
- All orientation support
- Harbor validation compliance
- RPM packaging and signing
- OpenRepos distribution
- Documentation
- Performance optimization
- Memory leak testing

Checklist:

- ☐ All Silica components used (no Qt Quick Controls)
- ☐ Active cover with actions
- ☐ Pull-down menus on all pages
- ☐ Proper Theme property usage
- ☐ Icon sizes: 86, 108, 128, 172
- ☐

- ☐ .desktop with Sailjail permissions
- ☐ RPM builds for all three architectures
- ☐ sfdk check passes
- ☐ Emulator testing
- ☐ Device testing (ARM64)

Distribution:

Sign RPM

```
sfdk engine exec rpmsign-external sign \  
    -k ~/.auroraos-regular-keys/regular_key.pem \  
    -c ~/.auroraos-regular-keys/regular_cert.pem \  
    RPMS/*.rpm
```

Validate

```
sfdk check RPMS/ru.auroraos.helixvpn-0.1.0-1.aarch64.rpm
```

Upload to OpenRepos

(manual upload via openrepos.net web interface)

Appendix A: Aurora OS Device Reference

Device	Screen	RAM	CPU	Architecture
F+ R570E	5.7" HD+	3 GB	SC9832E	armv7hl
Aquarius CMP NS	5.5" FHD	4 GB	MSM8937	aarch64
INOI R7	6.5" HD+	4 GB	T610	aarch64
Aurora Center	6.7" FHD+	6 GB	T618	aarch64

Appendix B: Debugging Commands

View ConnMan VPN logs

```
devel-su journalctl -u connman-vpn -f
```

List VPN connections

```
connmanctl vpnconnections
```

Check D-Bus services

```
qdbus --system net.connman.vpn /
```

```

# Monitor D-Bus traffic
busctl monitor --system net.connman.vpn

# Check app logs
devel-su journalctl --user -t ru.auroraos.helixvpn -f

# Verify Rust library loading
ldd /usr/bin/ru.auroraos.helixvpn
ls -la /usr/share/ru.auroraos.helixvpn/lib/

```

Appendix C: File Structure

```

helix-aurora/
├── helix-aurora.pro           # qmake project file
├── CMakeLists.txt           # CMake alternative
├── rpm/
│   └── ru.auroraos.helixvpn.spec  # RPM spec
├── src/
│   ├── main.cpp             # Entry point
│   ├── helixffi.h/cpp       # C++ Rust bridge
│   ├── connmandbus.h/cpp    # ConnMan D-Bus client
│   ├── vpncontroller.h/cpp  # QML controller
│   ├── serverlistmodel.h/cpp # Server list model
│   ├── settingsmanager.h/cpp # Settings persistence
│   ├── notificationsservice.h/cpp # Notifications
│   └── ffi/
│       └── helix_aurora.h    # Generated C header
├── qml/
│   ├── main.qml             # Root ApplicationWindow
│   ├── pages/
│   │   ├── MainPage.qml    # Connection UI
│   │   ├── ServerListPage.qml # Server selection
│   │   ├── SettingsPage.qml # Preferences
│   │   ├── SplitTunnelPage.qml # Split tunneling
│   │   └── AboutPage.qml    # About dialog
│   └── cover/
│       └── CoverPage.qml    # Active cover
├── icons/
│   ├── 86x86/
│   ├── 108x108/
│   ├── 128x128/
│   └── 172x172/
├── ru.auroraos.helixvpn.desktop # Desktop entry
├── ru.auroraos.helixvpn.permissions # Sailjail permissions
└── translations/
    └── helix-vpn-ru.ts      # Russian translation

```

Document version: 1.0

Platform: Aurora OS 4.x / 5.x (Sailfish OS derivative)

Architecture: Qt6/QML + C++ + Rust Core via FFI

Distribution: OpenRepos / Aurora Store (Harbor)

Estimated code reuse from Rust core: ~75%